example, the UNIX file system stores permissions for each file as a simple triple of permissions for the current user, the user's group, and anyone else. Rather than storing permissions for many individual users, the owner of the file can assign permissions to only a single pre-existing group, dramatically reducing the amount of data that must be stored for each file. The downside of this compression is that if a group doesn't exist with the required members, then the owner may have to grant access to a larger group than they would otherwise like to.

The implementation of simple groups is straightforward. Currently in the Natter API you have written, there is a `users` table and a `permissions` table that acts as an ACL linking users to permissions within a space. To add groups, you could first add a new table to indicate which users are members of which groups:

```
CREATE TABLE group_members(
    group_id VARCHAR(30) NOT NULL,
    user_id VARCHAR(30) NOT NULL REFERENCES users(user_id));
CREATE INDEX group_member_user_idx ON group_members(user_id);
```

When the user authenticates, you can then look up the groups that user is a member of and add them as an additional request attribute that can be viewed by other processes. Listing 8.1 shows how groups could be looked up in the `authenticate()` method in `UserController` after the user has successfully authenticated.

Listing 8.1 Looking up groups during authentication